

Formal Verification: 6CCS3VER

Exercise Sheet 8

Exercise 8.1 Fold function

The fold function is a very generic function, that can be used to express multiple other interesting functions over lists.

Have a look at Isabelle/HOL's standard function *fold*, defined as follows:

```
fun fold:: "('a ⇒ 'b ⇒ 'b) ⇒ 'a list ⇒ 'b ⇒ 'b" where  
  "fold f [] acc = acc"  
| "fold f (x # xs) acc = fold f xs (f x acc)"
```

Write a function to compute the sum of the elements of a list. Define two versions, one direct recursive definition, and one using fold. Show that both are equal.

Hint: use automation!

```
fun list_sum :: "nat list ⇒ nat"
```

```
definition list_sum' :: "nat list ⇒ nat"
```

```
lemma "list_sum xs = list_sum' xs"
```

Exercise 8.2 Tail recursive *reverse*

In Isabelle, there is the reverse function *rev*, which, given a list, computes another list with the same elements but in reverse order. Take a look into its definition:

```
thm rev.simps
```

Recall tail recursion: a function is tail recursive if in all recursive calls it does not call anything after itself¹. For instance, the implementation of *rev* in Isabelle is not tail recursive, because it calls (@) after it calls *rev* in the second equation.

Write an alternative implementation of the reverse function that is tail recursive.

¹See https://en.wikipedia.org/wiki/Tail_call. for a quick review.

Hint: recall that you can most of the times derive a tail recursive function using an accumulator argument.

fun *rev_tr*::*"'a list* $\Rightarrow$ *'a list"* **where**

Show that the two implementations are equivalent.

lemma *"rev_tr xs = rev xs"*